

Task 1

Docker & REST

Overview

A Basic REST Service was implemented to show the working of CRUD commands (GET, POST, PUT and DELETE). Python + Flask framework was used to implement this. The requests and responses were returned using JSON. The service was initially hosted locally and tested. Then a container was created using a docker file which was built and run. Now the same service was hosted within this container and its functionality was tested. Verification was done using the 'curl' command. The repository was pushed to git.

Github Repo: <https://github.com/bavanmendon/dsa-practicals/tree/master>

Code

- The array 'items' is used to store test content.

```
# In-memory data store
items = ['Apple', 'Banana', 'Carrot']
```
- This function is used to find the item by ID.

```
# Helper: Find item by id
def find_item(item_id):
    return next((item for item in items if item['id'] == item_id), None)
```
- This function is used to list all items.

```
# GET /items - List all items
@app.route('/items', methods = ['GET'])
def get_items():
    return jsonify(items), 200
```
- This function is used to retrieve specific content.

```
# GET /items/<id> - Retrieve specific item
@app.route('/items/<int:item_id>', methods = ['GET'])
def get_item(item_id):
    item = find_item(item_id)
    if item is None:
        return jsonify({'error': 'Item not found'}), 404
    return jsonify(item), 200
```
- This function was used to create a new item.

```
# POST /items - Create a new item (auto-increment the ID)
@app.route('/items', methods = ['POST'])
def create_item():
    global next_id
    data = request.get_json()
    if not data or 'name' not in data:
        return jsonify({'error': 'Name is required'}), 400
    item = {'id': next_id, 'name': data['name']}
    items.append(item)
    next_id += 1
```

```
return jsonify(item), 201
```

- This function was used to update an existing item.

```
# PUT /items/<id> - Update an existing item
@app.route('/items/<int:item_id>', methods = ['PUT'])
def update_item(item_id):
    item = find_item(item_id)
    if item is None:
        return jsonify({'error': 'Item not found'}), 404
    data = request.get_json()
    if not data or 'name' not in data:
        return jsonify({'error': 'Name is required'}), 400
    item['name'] = data['name']
    return jsonify, 200
```

- Finally, this function was used delete an item.

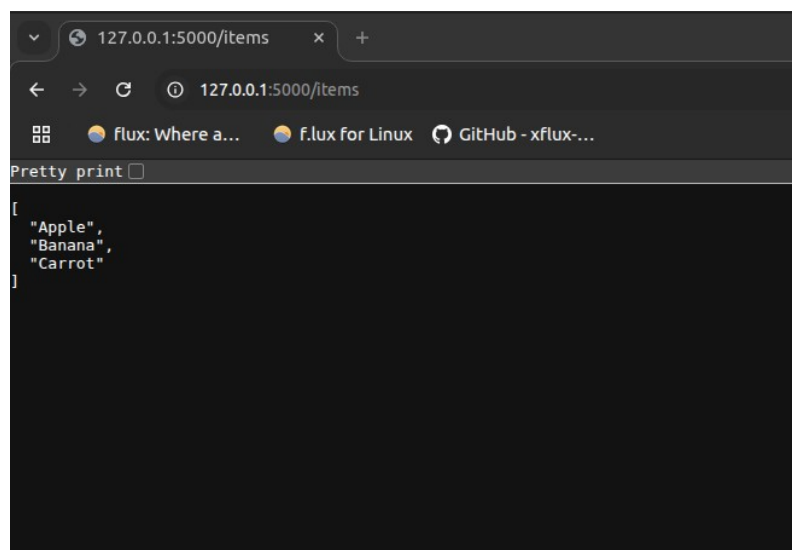
```
# DELETE /items/<id> - Delete item
@app.route('/items/<int:item_id>', methods = ['DELETE'])
def delete_item(item_id):
    item = find_item(item_id)
    if item is None:
        return jsonify({'error': 'Item not found'}), 404
    items.remove(item)
    return jsonify({'message': 'Item deleted'}), 200
```

- This section means that the service will be hosted on port 5000 and can be accessed by any IP.

```
if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000, debug = True)
```

Verification

When hosted locally and in the container, the dataset of 'items' was shown as below:



In the container, the functionality of the service was tested out using 'curl'. The screenshots are as given below:

This command displayed the contents of 'items'.

```
bavannendon@Electrofin:~$ curl http://localhost:5000/items
[
  "Apple",
  "Banana",
  "Carrot"
]
```

Now, the contents of items is initialised as NULL and two names are posted using 'curl'.

```
127.0.0.1:5000/items
flux: Where a... flux for Linux GitHub · xflux...
Pretty print
{
  "id": 1,
  "name": "Bavan"
},
{
  "id": 2,
  "name": "Prapti"
}
}

response = self.handle_exception(e)
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1511, in wsgi_app
response = self.full_dispatch_request()
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 920, in full_dispatch_request
return self.finalize_request(rv)
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 939, in finalize_request
response = self.make_response(rv)
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1241, in make_response
raise TypeError(
File "/usr/local/lib/python3.10/site-packages/flask/app.py", line 1236, in make_response
rv = self.response_class.force_type(
File "/usr/local/lib/python3.10/site-packages/werkzeug/wrappers/response.py", line 237, in force_type
response = Response(*run_wsgi_app(response, environ))
File "/usr/local/lib/python3.10/site-packages/werkzeug/test.py", line 1264, in run_wsgi_app
app_rv = app(environ, start_response)
File "/usr/local/lib/python3.10/site-packages/flask/json/_init_.py", line 170, in jsonify
return current_app.json.response(*args, **kwargs) # type: ignore[return-value]
File "/usr/local/lib/python3.10/site-packages/flask/json/provider.py", line 214, in response
f'(self.dumps(obj, **dump_args))\n', mimetype=self.mimetype
File "/usr/local/lib/python3.10/site-packages/flask/json/provider.py", line 179, in dumps
return json.dumps(obj, **kwargs)
File "/usr/local/lib/python3.10/json/_init_.py", line 238, in dumps
**kw).encode(obj)
File "/usr/local/lib/python3.10/json/encoder.py", line 201, in encode
chunks = list(chunks)
File "/usr/local/lib/python3.10/json/encoder.py", line 429, in _iterencode
yield from _iterencode_list(o, _current_indent_level)
File "/usr/local/lib/python3.10/json/encoder.py", line 325, in _iterencode_list
yield from chunks
File "/usr/local/lib/python3.10/json/encoder.py", line 405, in _iterencode_dict
yield from chunks
File "/usr/local/lib/python3.10/json/encoder.py", line 438, in _iterencode
o = _default(o)
File "/usr/local/lib/python3.10/site-packages/flask/json/provider.py", line 121, in _default
raise TypeError(f'Object of type {type(o).__name__} is not JSON serializable')
TypeError: Object of type builtin_function_or_method is not JSON serializable
The view function did not return a valid response. The return type must be a string, dict, list, tuple or
th headers or status, Response instance, or WSGI callable, but it was a function.

-->
bavannendon@Electrofin:~$ ^C
bavannendon@Electrofin:~$ curl -X POST -H "Content-Type: application/json" -d '{"name": "Bavan"}'
http://localhost:5000/items
{
  "id": 1,
  "name": "Bavan"
}
bavannendon@Electrofin:~$ curl -X POST -H "Content-Type: application/json" -d '{"name": "Prapti"}'
http://localhost:5000/items
{
  "id": 2,
  "name": "Prapti"
}
bavannendon@Electrofin:~$
```

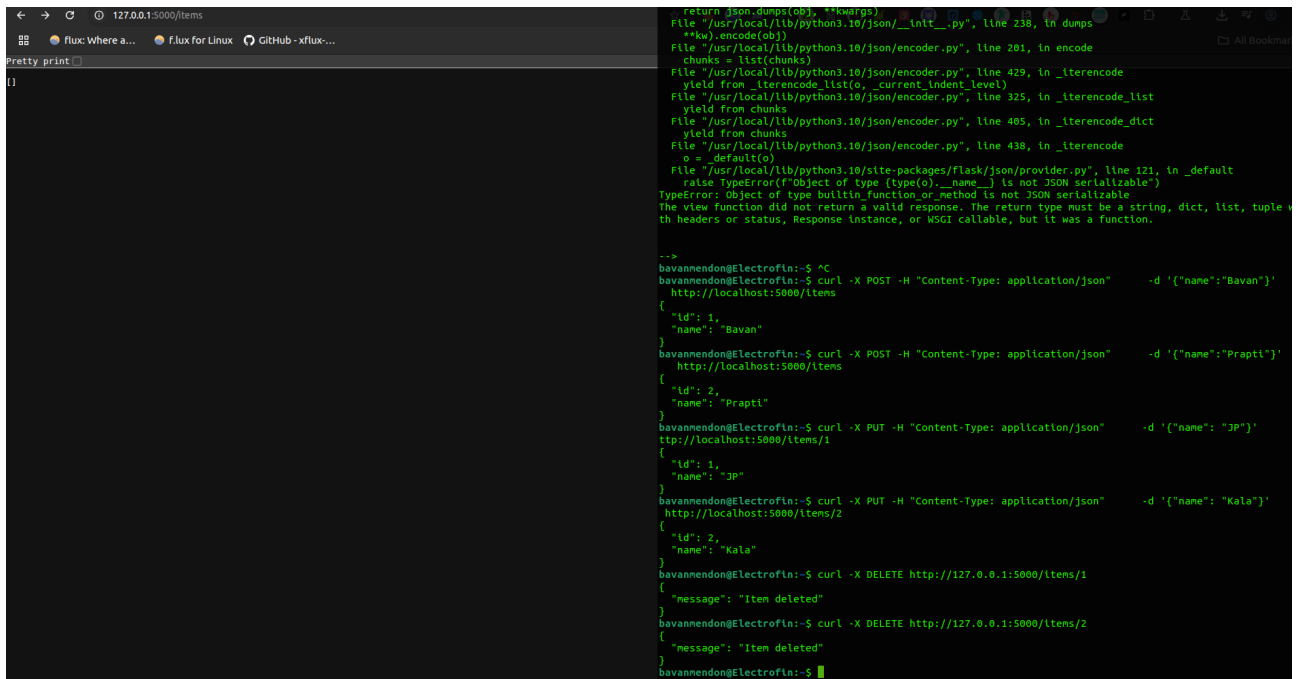
Now the contents of the array are updated using 'curl'.

```
127.0.0.1:5000/items
flux: Where a... flux for Linux GitHub · xflux...
Pretty print
{
  "id": 1,
  "name": "JP"
},
{
  "id": 2,
  "name": "Kala"
}
}

response = Response(*run_wsgi_app(response, environ))
File "/usr/local/lib/python3.10/site-packages/werkzeug/test.py", line 1264, in run_wsgi_app
app_rv = app(environ, start_response)
File "/usr/local/lib/python3.10/site-packages/flask/json/_init_.py", line 170, in jsonify
return current_app.json.response(*args, **kwargs) # type: ignore[return-value]
File "/usr/local/lib/python3.10/site-packages/flask/json/provider.py", line 214, in response
f'(self.dumps(obj, **dump_args))\n', mimetype=self.mimetype
File "/usr/local/lib/python3.10/site-packages/flask/json/provider.py", line 179, in dumps
return json.dumps(obj, **kwargs)
File "/usr/local/lib/python3.10/json/_init_.py", line 238, in dumps
**kw).encode(obj)
File "/usr/local/lib/python3.10/json/encoder.py", line 201, in encode
chunks = list(chunks)
File "/usr/local/lib/python3.10/json/encoder.py", line 429, in _iterencode
yield from _iterencode_list(o, _current_indent_level)
File "/usr/local/lib/python3.10/json/encoder.py", line 325, in _iterencode_list
yield from chunks
File "/usr/local/lib/python3.10/json/encoder.py", line 405, in _iterencode_dict
yield from chunks
File "/usr/local/lib/python3.10/json/encoder.py", line 438, in _iterencode
o = _default(o)
File "/usr/local/lib/python3.10/site-packages/flask/json/provider.py", line 121, in _default
raise TypeError(f'Object of type {type(o).__name__} is not JSON serializable')
TypeError: Object of type builtin_function_or_method is not JSON serializable
The view function did not return a valid response. The return type must be a string, dict, list, tuple or
th headers or status, Response instance, or WSGI callable, but it was a function.

-->
bavannendon@Electrofin:~$ ^C
bavannendon@Electrofin:~$ curl -X POST -H "Content-Type: application/json" -d '{"name": "Bavan"}'
http://localhost:5000/items
{
  "id": 1,
  "name": "Bavan"
}
bavannendon@Electrofin:~$ curl -X POST -H "Content-Type: application/json" -d '{"name": "Prapti"}'
http://localhost:5000/items
{
  "id": 2,
  "name": "Prapti"
}
bavannendon@Electrofin:~$ curl -X PUT -H "Content-Type: application/json" -d '{"name": "JP"}'
http://localhost:5000/items/1
{
  "id": 1,
  "name": "JP"
}
bavannendon@Electrofin:~$ curl -X PUT -H "Content-Type: application/json" -d '{"name": "Kala"}'
http://localhost:5000/items/2
{
  "id": 2,
  "name": "Kala"
}
bavannendon@Electrofin:~$
```

Finally, the contents of the array are deleted.



```
return json.dumps(obj, **kwargs)
File "/usr/local/lib/python3.10/json/__init__.py", line 238, in dumps
  **kw).encode(obj)
File "/usr/local/lib/python3.10/json/encoder.py", line 201, in encode
  chunks = list(chunks)
File "/usr/local/lib/python3.10/json/encoder.py", line 429, in _iterencode
  yield from _iterencode_list(o, _current_indent_level)
File "/usr/local/lib/python3.10/json/encoder.py", line 325, in _iterencode_list
  yield from chunks
File "/usr/local/lib/python3.10/json/encoder.py", line 405, in _iterencode_dict
  yield from chunks
File "/usr/local/lib/python3.10/json/encoder.py", line 438, in _iterencode
  o = _default(o)
File "/usr/local/lib/python3.10/site-packages/flask/json/provider.py", line 121, in _default
  raise TypeError(f'Object of type {type(o).__name__} is not JSON serializable')
TypeError: Object of type builtin_function_or_method is not JSON serializable
The view function did not return a valid response. The return type must be a string, dict, list, tuple
th headers or status, Response instance, or WSGI callable, but it was a function.

-->
bavannendong@ElectroFtn:~$ ^C
bavannendong@ElectroFtn:~$ curl -X POST -H "Content-Type: application/json" -d '{"name": "Bavan"}'
http://localhost:5000/items
{
  "id": 1,
  "name": "Bavan"
}
bavannendong@ElectroFtn:~$ curl -X POST -H "Content-Type: application/json" -d '{"name": "Prapti"}'
http://localhost:5000/items
{
  "id": 2,
  "name": "Prapti"
}
bavannendong@ElectroFtn:~$ curl -X PUT -H "Content-Type: application/json" -d '{"name": "JP"}'
http://localhost:5000/items/1
{
  "id": 1,
  "name": "JP"
}
bavannendong@ElectroFtn:~$ curl -X PUT -H "Content-Type: application/json" -d '{"name": "Kala"}'
http://localhost:5000/items/2
{
  "id": 2,
  "name": "Kala"
}
bavannendong@ElectroFtn:~$ curl -X DELETE http://127.0.0.1:5000/items/1
{
  "message": "Item deleted"
}
bavannendong@ElectroFtn:~$ curl -X DELETE http://127.0.0.1:5000/items/2
{
  "message": "Item deleted"
}
bavannendong@ElectroFtn:~$
```

Conclusion

The task requirements were fulfilled and verified using CRUD commands sent using the command 'curl'.